RepoMem: Measuring and Repairing the Working-Memory Bottleneck in Small Gemma Code Agents

A distance-controlled code-memory benchmark, and a graph-retrieval buffer that brings a 2B model to its oracle

Track: Gemma 4 Developer Agent — Paper Track · **Code:** https://github.com/leomaxuz/repomem (Apache-2.0) · companion Kaggle notebook

---

Abstract

Small quantized models such as Gemma 4 E2B and E4B are attractive for local developer agents. But on real repositories, the fact needed to answer a question may sit thousands of tokens away from the question, or not fit in the context window at all. We introduce **RepoMem**, a benchmark generated automatically from the competition's code graphs. It holds a question and its answer fixed and varies only the amount of distractor code between them (`distance_tokens`). RepoMem has 1,554 well-posed, exactly scorable questions of three kinds (`defn`, `chain`, `caller`) over four Python libraries.

RepoMem separates **two failure regimes**. *Within the window*, accuracy decays with distance even though the model holds the knowledge: shown only the needed function, the same model answers the same question at a flat 89–93%. The decay is strongly size-dependent. E2B falls from 90.9% to 71.7% at 8–20K tokens, 20.7 points below its own oracle. Larger models lose far less (overall gap 9.3 points for E2B versus 0.7–4.7 for E4B, 12B and 31B). *Beyond the window*, 2.7% of prompts exceed 32K tokens, and no model can answer them from raw context. Thinking mode does not help.

We then propose **PFK-agent**, a working-memory architecture. It starts from the entity named in the question, walks the code graph with the neighbor and caller tools a Gemma developer agent already has, and answers from a small buffer. PFK-agent lifts E4B from 85.7% to 89.1% and **E2B from 79.8% to 91.3%**, matching E2B's oracle (91.4%). That puts E2B above the 12B baseline at 1/52 of 31B's latency. PFK-agent also answers 81–88% of the beyond-window questions. With a full-context fallback, it stays robust when the model grounds the entity itself (88.5%) or when 30% of graph edges are missing (88.4%).

Finally, we release **RepoMem-OSS**, a fully open companion benchmark built from six public repositories with our own AST graph builder. The pattern holds there too: E2B's single-hop accuracy falls from 91.7% to 65.0% with distance, and PFK-agent restores it to 93.3%.

---

1. Introduction

A developer agent built on a small model rarely fails because it lacks knowledge of a function. More often the function is buried under unrelated code, or lies outside the context entirely. Gemma 4's small models rely mostly on sliding-window attention, which is efficient but limits how reliably they use distant context.

Generic probes plant a random sentence in prose. For code agents, the needle is a function, the distractors are plausible sibling functions, and the question concerns a relationship. We contribute:

1. **RepoMem**, an automatic, distance-controlled, exactly scorable benchmark built from code graphs. Its well-posedness filter exposes reversed `calls` edges in the source graphs (§3). We also release **RepoMem-OSS**, a fully open companion built from six public repositories (§6.4).
2. A **measurement** of two regimes: within-window decay, which is far worse for the smallest model, and a hard beyond-window ceiling that scale does not remove. An oracle control shows the knowledge is present (§5).
3. **PFK-agent**, a tool-grounded working-memory buffer that recovers the oracle without oracle information. We ablate entity grounding, graph noise, and routing (§6).

---

2. Related Work

Long context. Liu et al. [1] find a U-shaped "lost in the middle" position effect, and needle-in-a-haystack tests [2] probe verbatim recall of one sentence. RepoMem uses a code-structured haystack, asks *relational* questions, and sweeps distance explicitly. Sliding-window attention [4] is the regime where a memory bottleneck should dominate.

Code benchmarks. RepoBench [6], CrossCodeEval [7], and SWE-bench [5] measure end-task success, which entangles retrieval, knowledge, and reasoning. RepoMem is diagnostic: it isolates memory.

Memory. Retrieval augmentation [3], external memory [10], and models of prefrontal working memory [9, 11] motivate a small, gated buffer. PFK-agent builds it from the agent's own graph tools rather than a vector index.

---

3. The RepoMem Benchmark

Source. Each competition code graph has nodes (function/class `id`, `name`, source `text`) and `calls` edges. We use all 127 unique graphs of fastapi, rich, requests, and httpx. Each item is a JSONL record `{id, repo, kind, k, distance_tokens, n_items, prompt, answer, meta}`. The prompt is a list of fenced function blocks followed by a question.

Probes. - `defn`: list the parameters of a named function among N shown. - `chain`: follow a call chain from a named function for exactly k steps and name the function reached. - `caller`: name the shown function that calls a named target.

Well-posedness. Every answer is unique and grounded in the shown code: - Every `chain` step has exactly one in-graph callee, and every `caller` target has exactly one in-graph caller. - Every step must be **visible as a call (`name(`) in the caller's source**. - Parameters come from the Python AST, excluding `self` / `cls`. - The verbatim question is stored, so every condition asks the identical question.

The visibility check matters: **97 of 508 chain candidates (19%) relied on a `calls` edge absent from the source**. In inspected cases the edge points callee → caller (the graph says `fake_password_hasher → fake_save_user`; the reverse is true). Their gold answers are unanswerable (oracle 37% vs 83% elsewhere). The generator drops them after sampling, leaving 1,554 items, which are identical with or without the filter. Scoring is deterministic, with no LLM judge.

Composition. `distance_tokens` (0 to ~33K) is bucketed 0–512 / 512–2K / 2K–8K / 8K–20K / 20K+ (369 / 360 / 501 / 283 / 41 items). There are 635 `defn`, 411 `chain`, and 508 `caller` items. **42 items (2.7%) have prompts longer than the 32K window**; we report them separately as *beyond-window*. The other 1,512 are *in-window*.

---

4. Setup

Models and serving. We evaluate the competition's int4 QAT checkpoints of the Gemma family [8], `gemma-4-{e2b,e4b,12b,31b}-it-qat-w4a16-ct`, on vLLM 0.33.0 with 2× RTX 4090 (tensor parallelism 2), greedy decoding, and thinking off unless stated. The window is `max_model_len = 32768`.

31B and the common subset. 31B's KV cache fits only 16K tokens on this hardware. Cross-size comparisons therefore use the **1,397 items that fit every model**.

Conditions. *Baseline* uses the full prompt. *Oracle (compact)* shows only the needle block(s) with the same question, an upper bound that uses ground truth. *PFK-agent* is described in §6.

---

5. Measuring the Bottleneck

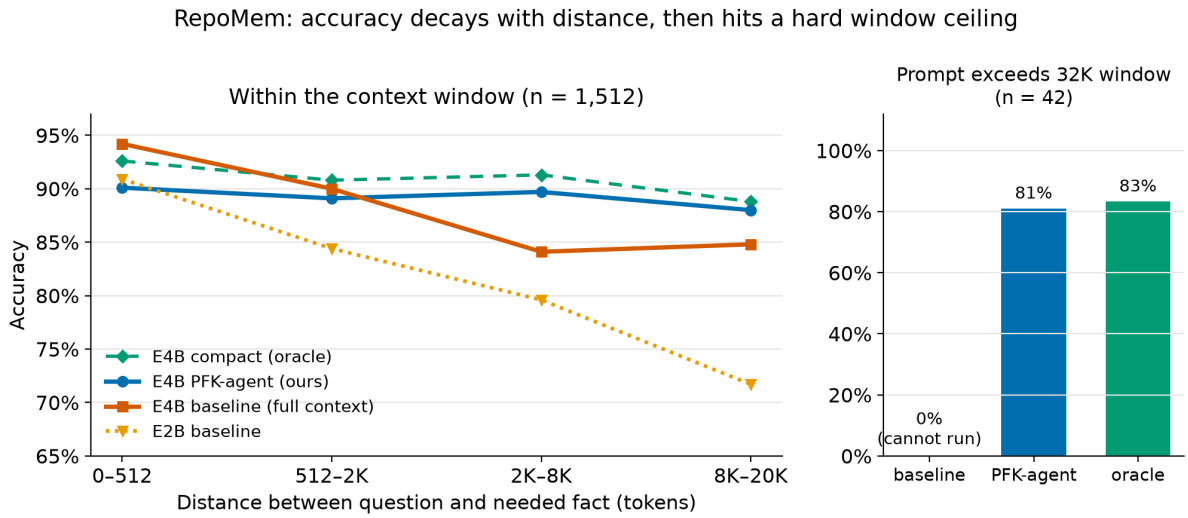


Figure 1. The two regimes on the main set.

5.1 Within the window: decay with distance, knowledge intact

distance	n	E4B base	E4B oracle	E2B base	E2B oracle
0–512	364	94.2	92.6	90.9	92.9
512–2K	359	90.0	90.8	84.4	88.0
2K–8K	496	84.1	91.3	79.6	92.5
8K–20K	276	84.8	88.8	**71.7**	92.4
20K+ (fits)	17	88.2	88.2	76.5	88.2

Shown only the needle, both models stay flat (88–93%), so the decay is not a knowledge or reasoning gap. The model fails to *retrieve* a fact it can use: a working-memory limit. E4B loses up to 7.2 points to its oracle. E2B loses **20.7 points** at 8–20K, even though its oracle (91.5%) matches E4B's (91.0%).

5.2 Scaling: the smallest model suffers most

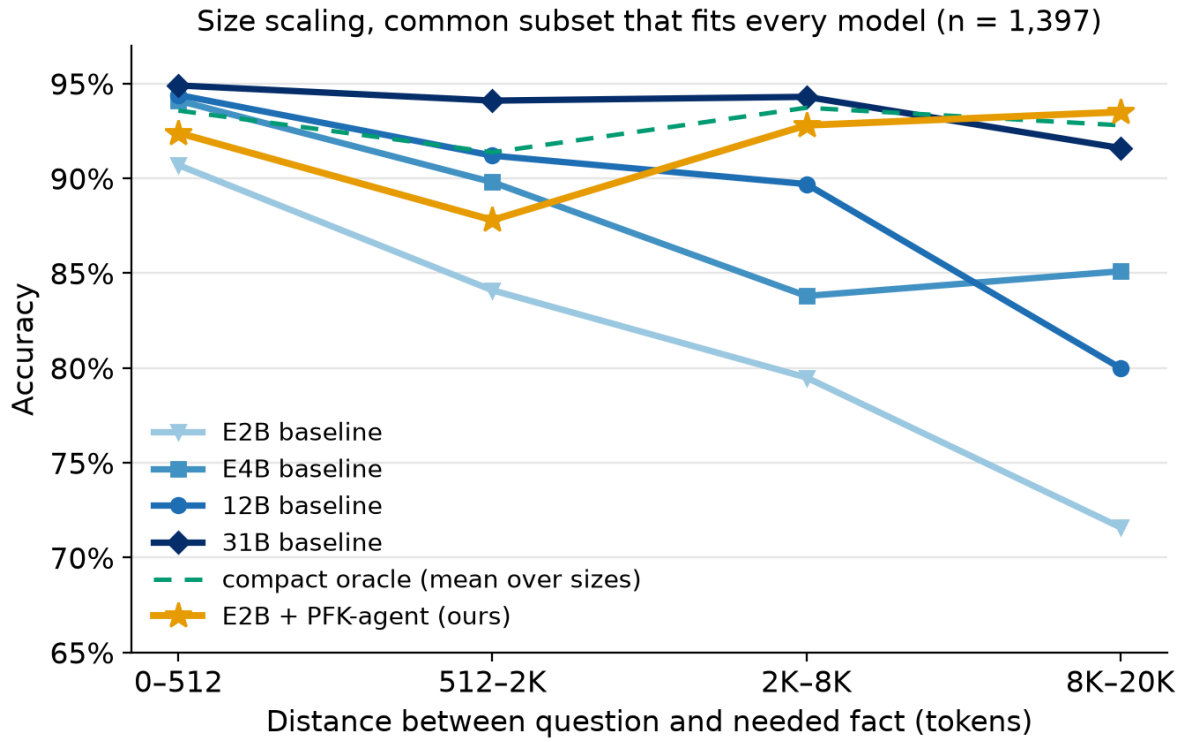


Figure 2. Four sizes on the 1,397 items that fit every model. The orange line is E2B with PFK-agent.

model	baseline	oracle	gap 2K–8K	gap 8K–20K	s/query
E2B	82.3	91.6	13.3	**21.9**	2.05
E4B	88.1	91.0	7.6	3.3	3.56
12B	89.8	94.5	4.8	13.5	6.91
31B	94.0	94.7	1.9	4.2	11.54
E2B + PFK	**91.6**	—	—	—	**0.22**

The oracle varies little with size (91.0–94.7%), but the distance penalty does not. E2B is the clear outlier (overall gap 9.3 points versus 2.9 / 4.7 / 0.7 for E4B / 12B / 31B). Above E2B the trend is not monotone: 12B, which uses a different (unified multimodal) architecture, is weak at 8–20K. The bottleneck is largest for the most deployable model.

5.3 Beyond the window: a ceiling scale cannot remove

On the 42 beyond-window items the baseline returns nothing. Larger models share the 32K window (31B fits only 16K here, leaving 157 items unservable), and RepoMem prompts reach ~110K tokens. Reading a small buffer, the oracle and PFK-agent answer these items at 83.3% and **81.0%** (E4B), and 88.1% and **88.1%** (E2B).

5.4 Thinking and position do not explain it

Thinking leaves accuracy unchanged (90.1% vs 89.8% on the same 353 sampled items). By the needle's relative position in long prompts, E4B scores 80.5 / 86.6 / **86.5** / 84.0 / 82.0%: the middle is *best*, so there is no lost-in-the-middle U-shape [1].

---

6. PFK-agent: A Graph-Retrieval Working Memory

Method. PFK-agent treats the entity named in the question as a handle into the repository. It builds a compact buffer with the operations exposed to a Gemma developer agent (`get_code_neighbors` / `get_code_subgraph`): - `defn` : fetch the named block. - `chain` : walk `calls` edges from the named start, one step at a time. - `caller` : follow reverse edges.

It answers from the buffer alone, never reading the stored answer path or the needle set.

6.1 Main result

	E4B base	**E4B PFK**	E4B oracle	E2B base	**E2B PFK**	E2B oracle
0–512	94.2	90.1	92.6	90.9	92.6	92.9
2K–8K	84.1	**89.7**	91.3	79.6	**92.5**	92.5
8K–20K	84.8	**88.0**	88.8	71.7	**92.4**	92.4
beyond window	0.0	**81.0**	83.3	0.0	**88.1**	88.1
all 1,554	85.7	**89.1**	90.8	79.8	**91.3**	91.4

Graph retrieval succeeds on 100% of items. On E4B, PFK recovers **67% of the oracle headroom** and comes within 0.8–2.3 points of the oracle wherever the bottleneck bites. At short distance it trails the baseline by 4.1 points, because the bare buffer drops useful surroundings.

On E2B, PFK **matches the oracle** in every bucket. On the common subset, E2B + PFK (91.6%) beats the E4B and 12B baselines (88.1%, 89.8%) and is 2.4 points below 31B (94.0%) at 1/52 of its latency (0.22 s vs 11.54 s). PFK is ~10× faster than the E4B baseline, with latency flat in distance.

By kind (E4B), `caller` gains most (93.3% → 98.4%, equal to the oracle), `chain` improves (76.4% → 83.2%), and `defn` is unchanged. Baseline failures typically name a *real, plausible neighbor* (e.g. `get_current_version` for `update_version_file`): interference, not missing knowledge.

6.2 Robustness across benchmark instances

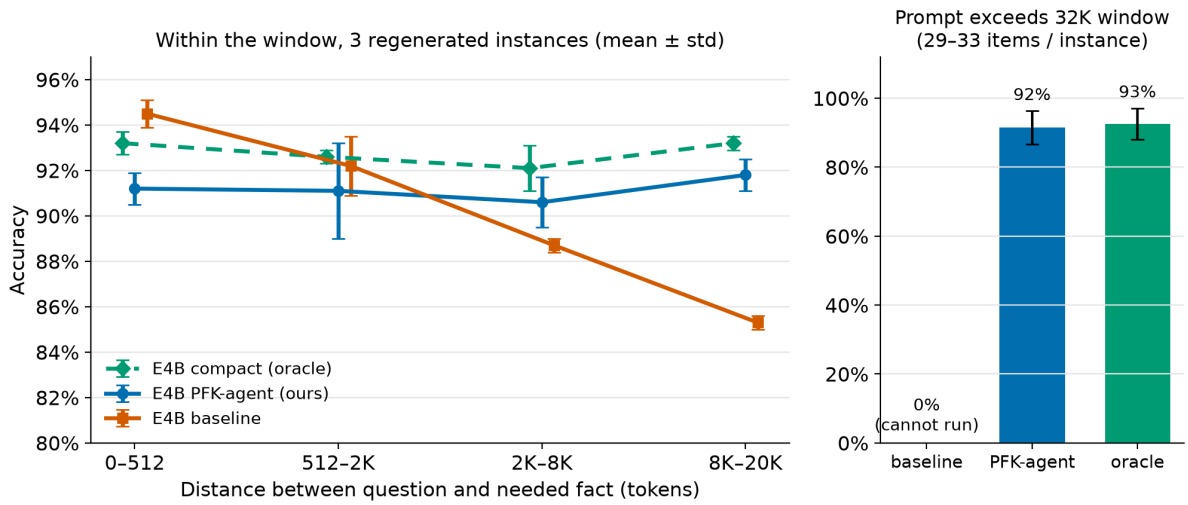


Figure 3. Mean ± std over three regenerated instances (seeds 1–3, 1,410–1,423 well-posed items each, E4B).

	0–512	512–2K	2K–8K	8K–20K	beyond	all
baseline	94.5±0.6	92.2±1.3	88.7±0.3	**85.3±0.3**	0.0	88.3±0.4
PFK	91.2±0.7	91.1±2.1	90.6±1.1	**91.8±0.7**	91.5±4.8	91.2±0.9
oracle	93.2±0.5	92.6±0.3	92.1±1.0	93.2±0.3	92.6±4.5	92.7±0.4

The baseline decays

monotonically

(94.5% → 85.3%) while PFK and the oracle stay flat. The 8–20K gap (6.5 points) is ~10× the across-instance standard deviation.

6.3 Ablations: grounding, graph noise, routing (E4B, all 1,554 items)

variant	accuracy
PFK, entity given	89.1
entity extracted by Gemma, naive name lookup	74.2
… + verification against the question + full-context fallback	**88.5**
only the bare function name visible (+ verification, fallback)	86.2
10% / 30% of `calls` edges removed	84.6 / 77.6
30% removed + full-context fallback when retrieval is incomplete	**88.4**
baseline (for reference)	85.7

Grounding. A naive lookup fails because Gemma often returns a bare name, and fastapi's tutorials repeat names like `get_user` across up to 45 modules. Accepting the answer only when it is unambiguous or literally in the question, with a full-context fallback otherwise, restores 88.5%.

Graph noise.

Missing edges break chain walks. The same fallback, triggered on 18% of items, recovers 88.4%.

Routing.

A prompt-length router, with its threshold chosen on seed 1 and frozen, does not beat PFK alone on held-out sets (−0.5 points).

6.4 Generalization: RepoMem-OSS

To test RepoMem beyond the competition, we wrote a stdlib-only builder (`build_graph.py`). It turns any Python repository into a code graph whose `calls` edges are resolved from call expressions in each caller's body. Applied to six public repositories absent from the competition (flask, click, werkzeug, jinja, attrs, and marshmallow, at pinned BSD/MIT releases), the same generator yields **1,534 items**, all in-window. The whole set is released openly.

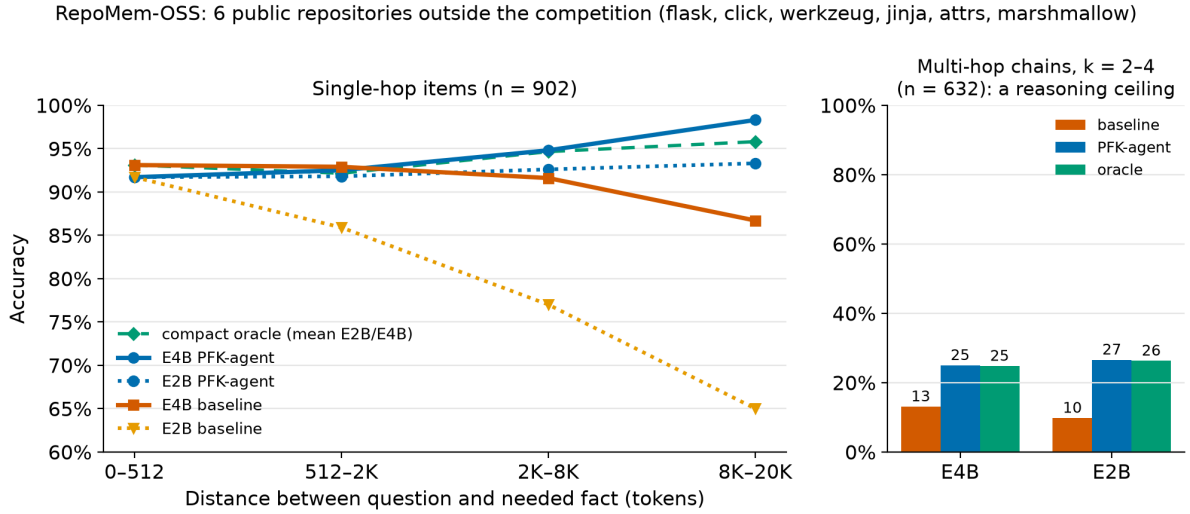


Figure 4. RepoMem-OSS. Left: single-hop items (n = 902). Right: multi-hop chains (n = 632).

single-hop (n = 902)	0–512	2K–8K	8K–20K	all
E2B base / **PFK** / oracle	91.7 / 91.7 / 93.1	77.0 / **92.6** / 93.5	**65.0** / **93.3** / 93.3	81.5 / **92.2** / 92.9
E4B base / **PFK** / oracle	93.1 / 91.7 / 93.1	91.6 / **94.8** / 95.8	86.7 / **98.3** / 98.3	91.7 / **93.9** / 94.6

On single-hop items (`defn`, `caller`, 1-step `chain`), every finding replicates: the baseline decays with distance, far more for E2B (28 points below the oracle at 8–20K), and PFK-agent tracks the oracle for both models.

Multi-hop chains (k = 2–4) behave differently. Even the oracle, shown only the chain, scores 25–26%. In inspected failures the gold chains were correct, but the models stopped after one hop or named an unshown callee. Here the limit is multi-hop *reasoning*, not memory. PFK still doubles the baseline (E4B 13.1% → 25.0%, E2B 9.8% → 26.6%), but it cannot exceed that ceiling.

---

7. Limitations

- **Graph as ground truth.** RepoMem is built from, and PFK retrieves over, the same graph. §6.3 bounds the effect of edge loss, but real failures are structured, not random.
- **Synthetic single-turn questions** isolate memory, but they are not natural developer requests and do not measure end-to-end agent success.
- **Small long-distance sets.** The 20K+-fitting bucket has 17 items and the beyond-window set 42. RepoMem-OSS has no beyond-window items.

- **Hardware-bounded scaling.** 31B is compared at a 16K window, and 12B differs architecturally.
- **Scope.** Ten Python repositories and three probe kinds, evaluated on E2B/E4B outside the competition data.

---

8. Conclusion

RepoMem turns "small models forget things far away" into a controlled measurement and separates two phenomena. Within the window, small Gemma models lose facts they demonstrably know, and the loss is by far the largest for E2B. Beyond the window, no model of any size can answer from raw context.

PFK-agent addresses both regimes. It is a minimal working-memory buffer built from the graph tools a Gemma agent already has. It brings the smallest model exactly to its oracle, above the 12B baseline, at 10–50× lower latency, and it stays robust to self-grounded entities and incomplete graphs. The same results hold on RepoMem-OSS, which we release openly. That benchmark also shows the next frontier: once memory is solved, multi-hop reasoning over code becomes the limit. Building RepoMem surfaced reversed edges in the competition graphs, a caution for agents that trust graph tools blindly.

Reproducibility

All code is released under Apache-2.0 at https://github.com/leomaxuz/repomem: - `generate.py`: the generator. `--per-repo 20 --seed 1` reproduces the 1,554 items. - `build_graph.py`: the code-graph builder for any Python repository. - `evaluate.py`: baseline, oracle, and thinking runs. - `pfk_agent.py`: PFK-agent and its ablation flags. - `recompute_all.py` and `ext_numbers.py`: compute every number in this paper.

RepoMem-OSS (items, graphs, manifest with commits, and source licenses) is in `benchmark/repomem-oss/`. A companion Kaggle notebook regenerates the competition benchmark, which is Competition Data and is not redistributed.

References

[1] Liu et al. Lost in the Middle: How Language Models Use Long Contexts. TACL 2024. [2] G. Kamradt. Needle In A Haystack — Pressure Testing LLMs. 2023. [3] Lewis et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS 2020. [4] Beltagy, Peters, Cohan. Longformer: The Long-Document Transformer. arXiv:2004.05150, 2020. [5] Jimenez et al. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? ICLR 2024. [6] Liu, Xu, McAuley. RepoBench: Benchmarking Repository-Level Code Auto-Completion Systems. ICLR 2024. [7] Ding et al. CrossCodeEval: A Diverse and Multilingual Benchmark for Cross-File Code Completion. NeurIPS 2023. [8] Gemma Team. Gemma: Open Models Based on Gemini Research and Technology. 2024. [9] A. Baddeley. Working Memory: Looking Back and Looking Forward. Nat. Rev. Neurosci. 2003. [10] Graves et al. Hybrid Computing Using a Neural Network with Dynamic External Memory. Nature 2016. [11] O'Reilly, Frank. Making Working Memory Work: A Computational Model of Learning in the Prefrontal Cortex and Basal Ganglia. Neural Computation 2006.